

矮人要塞本体架构与设计理念

《矮人要塞》由Tarn Adams一人主导开发，核心代码使用C和C++混合编写¹。游戏引擎基于SDL+OpenGL实现跨平台图形显示，使用FMOD处理音频¹。架构上是一个高度耦合的单一代码库，没有采用多线程（除渲染外完全单线程）²。设计风格是持续演进式开发：随着20多年功能增量，代码结构较为“杂乱”（legacy和补丁堆积），Tarn会写大量注释和改名以便日后维护³⁴。例如，他曾将物品系统从多态类层次改为组件式结构（接近ECS思想），以改进性能和灵活性⁵。储存方面，存档采用二进制格式，并在存档头记录各数据块大小，保证跨平台/大小端兼容⁴。路径规划使用A*算法，并动态维护连通分量来优化路径查询，避免频繁全地图路径搜索⁶。

核心机制程序设计概述

- **世界生成**：世界生成分多个阶段（如生成地形高程、温度、河流、湖泊与矿物、生物群落、文明分布、地下洞穴文明和地下生物、巨兽、遗迹等）⁷。过程使用伪随机数（PRNG）和种子保证可复现性，参数调整通过多级网格加噪声（weighted mesh + noise）来塑造环境分布⁸⁹。游戏中还采用“拒绝采样”策略：如果生成结果不符合某些约束（如资源分布、气候条件等），则重算新世界¹⁰。
- **路径与AI**：生物移动由A*寻路实现，Tarn维护了区域连通性标记以避免每次都完整寻路⁶。AI行为基于事件驱动的任务队列（如生产、采集、作战等），每个实体按优先级执行有限状态机式的行为，但具体算法未公开。游戏是一个滴答式仿真，每个游戏刻钟更新所有实体状态和环境。
- **社会与经济模拟**：每个矮人拥有属性、心情、特质、家族和记忆等状态，这些通过数据结构存储，每个刻钟更新（如幸福度、社交互动等）。经济方面模拟生产链与贸易，但具体实现细节不透明，只能推测为基于资源队列和事件驱动的系统。
- **战斗与伤害**：没有传统HP系统，战斗伤害采用高保真模型：每个生物由多个部位（如头、躯干、四肢等）和各自的层级（皮肤、肌肉、骨骼、血管、内部器官等）组成¹¹。当受到伤害时，伤害穿透各层并记录到具体部位，导致瘀伤、骨折、失血、器官破裂等效果¹¹。例如，钝击会造成骨折和内伤，锋利武器易割开血管，穿刺武器穿透能力强等。战斗结果通过详细文本报告展示。
- **流体物理**：流体（仅水和熔岩）按简单物理规则模拟：流体会优先向下流动，其次水平扩散（支持同层对角流动，但不同时进行下落和横向）¹²。此外，水具有“压力”机制：当高处落下的7/7（水满格）水柱下方为满水时，可沿已满水连通路“瞬移”到远处空格，实现快速灌满系统¹³。熔岩则无压力量特性。流体更新在每刻钟进行，模拟简单的重力和压力传播。
- **其他机制**：游戏还模拟结构支撑、地下塌陷、地形侵蚀等物理现象，以及文明历史事件、社会冲突等高级模拟，这些往往以事件日志形式在世界生成后进行，并在传奇模式中体现。

社区工具架构与实现

- **DFHack**：DFHack是一个注入式内存访问库，用于读写游戏内存并扩展功能。它以C++编写，通过编译成动态库（如libdfhooks）被游戏加载，钩住游戏初始化和主循环点¹⁴。DFHack使用df-structures仓库中的XML定义了游戏内部数据布局，并在构建时生成对应的C++头文件¹⁵。它将功能按模块组织（如Units模块、Maps模块等），为插件提供接口¹⁶¹⁷。插件以C++实现，生成可在DFHack控制台调用的新命令；此外DFHack提供Lua层API，使脚本可以访问游戏结构和内置UI组件¹⁶¹⁷。DFHack还支持远程接口，外部工具可通过网络协议连接来操控游戏¹⁸。整体架构为：游戏进程加载DFHack库后，DFHack内核提供控制台、调试命令和模块，插件通过模块或脚本调用DF内存数据。
- **Dwarf Therapist**：Dwarf Therapist是独立运行的矮人管理工具，使用C++和Qt5（QtWidgets/QML）开发¹⁹。其核心功能是读取正在运行的《矮人要塞》进程内存，对照内存布局（由DFHack可导出）解析矮人属性、职业、工作优先级等信息，并以可排序、过滤的表格界面呈现²⁰。用户可以在

界面修改劳动力分配，DT将修改写回游戏内存，实现“批量”操作。它无需内嵌于游戏，通过系统调用（如Windows的进程读写或Linux的ptrace）与游戏进程通讯。更新新游戏版本时，DFHack提供“export-dt-ini”命令生成新内存布局，DT加载对应INI文件即可适配新版本²¹。

- **Legends Viewer**：原Legends Viewer是展示世界历史数据的工具，新版本LegendsViewer-Next采用现代架构：后端用.NET 8（C#）编写，前端用Vue3+TypeScript构建Web界面²²。后端负责快速解析由DFHack导出的传奇数据XML（使用高效的XmlReader读取大文件）并提供API，前端使用Leaflet.js绘制互动地图、展示文明和历史事件信息²³。解析后的世界数据可存为“书签”以便快速访问²⁴。
- **Legends Browser**：Legends Browser（尤其是新版）也是传奇数据查看器。LegendsBrowser2由Go语言编写，运行时启动本地Web服务器，用户在浏览器访问<http://localhost>即可查看界面²⁵。它将传奇XML数据转换为关联式页面（人物、地点、事件互链）并提供统计信息。使用时需要先在DFHack传奇模式下导出 `legends.xml` 和 `legends_plus.xml`²⁶。由于Go编译后的可执行文件体积小、跨平台，LegendsBrowser2无需其他依赖即可运行，只要加载XML即可。

开源仿制/兼容项目架构与设计

- **SharpFortress (C#)**：一个部分完成的开源仿制项目，目标是多线程高性能仿制DF，采用C#编写²⁷。它从一开始设计为多线程架构，以避免后期重构造成的性能瓶颈²⁷。项目采用与Bethesda类似的模块化结构（主程序+基础模组），核心模拟逻辑在引擎层面实现，细节尚在开发中。
- **Fortress (Rust)**：JosephLenton开发的Rust语言DF克隆，采用monorepo结构，代码按功能拆分为多个crate²⁸²⁹。主要组件包括：`game`（无图形的核心仿真引擎）、`world`（游戏数据定义，如生物、地形、物品等）、`generate`（世界/地图生成逻辑）、`head`（高层渲染管理器，控制视图位置和摄像机）、`llr`（低级渲染接口及SDL2/终端具体实现）等³⁰³¹。这种分层允许将模拟逻辑与渲染解耦：`game`产生模拟结果，`llr_sd12`则实际调用SDL2画图。模块化设计使得引擎、图形和工具互不混杂，语言选用Rust以获得安全性和性能（项目语言统计显示96.8%为Rust）³²。
- **Èrthur (JavaScript + Lua)**：Èrthur是一个导出并可视化DF地图的小工具集合³³。它包含一个DFHack Lua脚本（`export-embark-map.lua`）用于在游戏中导出当前世界的详细地图数据（生成`map.json`）³⁴。导出后，使用Node.js编写的`draw.js`脚本读取该JSON和颜色文件，将数据渲染为可缩放的高分辨率地图（生成HTML/Canvas图像）³⁴。该项目主要以JavaScript、HTML和Lua实现，借助DFHack和Node生态快速原型化地图可视化。
- **Armok Vision (Unity/C#)**：Armok Vision是一个实时3D可视化器，将《矮人要塞》的游戏状态映射为三维世界。它基于Unity引擎（使用C#），通过DFHack提供的接口实时获取游戏场景和实体数据。在架构上，它为Unity增添了DFHack插件，使得游戏地图格点可在3D中渲染（类似Minecraft风格）。由于使用Unity，其模块以游戏对象和脚本形式组织，支持VR输入设备。Armok Vision需要联动DFHack并在进入要塞模式后启动，即可“镜像”地面布局和实体行为。³⁵（项目已分发预编译版）。
- **LegendsBrowser2 (Go/HTML)**：新版LegendsBrowser（前文提及）由Go编写后端，内置HTTP服务，前端界面为HTML/JavaScript²⁵。后端解析XML，前端动态呈现数据。用户只需运行可执行文件并用浏览器打开对应地址，即可跨平台使用。该工具重写自Java版原LegendBrowser（RobJanetzko作品），强调轻量和扩展性，编译器仅需Go环境即可构建²⁵。

上述工具和项目大多开源于GitHub或社区论坛，涉及语言多样（C/C++、C#、Rust、Go、JavaScript、Lua等），架构设计上普遍采用模块化组织：如DFHack按功能拆模块，Fortress将引擎/渲染分层，LegendsViewer/Browser分离前后端，仿制项目则尝试引入并行和现代框架，以提升可维护性和性能。它们的接口方式包括：内存读取（DFHack/DT）、XML数据导入（Legends工具）、网络服务（LegendsBrowser）等，各具特色地复现或扩展了《矮人要塞》的功能。

参考资料：有关DF本体结构和设计的采访与解析¹⁶，DF世界生成与模拟机制的社区资料⁷⁸¹¹³⁶，以及DFHack、社区工具和开源项目的官方文档与仓库（如DFHack架构文档¹⁴¹⁶、Dwarf Therapist和Legends Viewer的GitHub介绍¹⁹²²，以及相关Wiki页面²⁰）。这些来源为推测与总结提供了依据和代码结构示例。

1 2 3 4 5 6 700,000 lines of code, 20 years, and one developer: How Dwarf Fortress is built - Stack Overflow

<https://stackoverflow.blog/2021/12/31/700000-lines-of-code-20-years-and-one-developer-how-dwarf-fortress-is-built/>

7 8 9 10 Advanced world generation - Dwarf Fortress Wiki

https://dwarffortresswiki.org/index.php/Advanced_world_generation

11 Combat - Dwarf Fortress Wiki

<https://dwarffortresswiki.org/index.php/Combat>

12 13 36 DF2014:Flow - Dwarf Fortress Wiki

<https://dwarffortresswiki.org/index.php/DF2014:Flow>

14 15 16 17 18 DFHack development overview — DFHack 50.13-r3 documentation

<https://docs.dfhack.org/en/50.13-r3/docs/dev/Dev-intro.html>

19 GitHub - Dwarf-Therapist/Dwarf-Therapist: Dwarf management tool for Dwarf Fortress

<https://github.com/Dwarf-Therapist/Dwarf-Therapist>

20 21 Utility:Dwarf therapist - Dwarf Fortress Wiki

https://dwarffortresswiki.org/index.php/Utility:Dwarf_therapist

22 23 24 GitHub - Kromtec/LegendsViewer-Next: Recreates Dwarf Fortress' Legends Mode from exported files.

<https://github.com/Kromtec/LegendsViewer-Next>

25 26 GitHub - robertjanetzko/LegendsBrowser2: Legends Browser is an multi-platform, open source legends viewer for dwarf fortress 0.47.

<https://github.com/robertjanetzko/LegendsBrowser2>

27 GitHub - dptetc/SharpFortress: An attempt at a well-performing dwarf fortress like game in C#

<https://github.com/dptetc/SharpFortress>

28 29 30 31 32 GitHub - JosephLenton/fortress: A Dwarf Fortress clone

<https://github.com/JosephLenton/fortress>

33 34 GitHub - nornagon/erthur: A mapping tool for Dwarf Fortress

<https://github.com/nornagon/erthur>

35 GitHub - RosaryMala/armok-vision: A 3d realtime visualizer for Dwarf Fortress

<https://github.com/RosaryMala/armok-vision>